

Network and API Access Controls

CompTIA SecAI+: AI Security Certification Complete Exam Prep 2026 | Section 12: Access Controls for AI Systems

1. Why AI Systems Require Dedicated Network Security

AI systems introduce network attack surfaces that traditional IT does not. An inference API exposes a queryable endpoint anyone can probe; a training pipeline ingests external data that could be poisoned; a model registry may synchronize across cloud regions over public links. Each of these is a potential entry point for adversaries seeking to intercept predictions, extract training data, corrupt model weights, or simply overwhelm the service. Treating AI infrastructure as a standard web application and applying only perimeter firewalls is insufficient. The CompTIA SecAI+ exam tests whether candidates understand the combination of network segmentation, zero-trust principles, API-layer controls, and continuous monitoring that together constitute a credible defense for AI workloads.

A practical starting point is a threat model: who calls your AI API, from where, how frequently, and what data moves in each direction? The answers drive every control decision that follows. A high-volume public threat-intelligence API has very different requirements from an internal, low-frequency model that scores fraud transactions. Map data flows first, then design controls around them.

2. Network Segmentation: Isolating AI Workloads by Risk

Segmentation divides a flat network into zones of differing trust and sensitivity. For AI environments, a three-tier model is common: a data ingestion and training zone (highest sensitivity, smallest blast radius), an inference and serving zone (public-facing or internal consumers), and a management and monitoring zone (log aggregation, MLOps tooling, model registry). East-west traffic between zones is restricted by firewall policy and, where possible, by cryptographic controls. This means even a fully compromised inference pod cannot directly query the training database or push a modified model to the registry.

Zone	Typical Assets	Inbound Permitted From	Outbound Permitted To
Training / Data	Raw datasets, training jobs, model artifacts	DevOps toolchain only	Model registry (push)
Inference / Serving	Inference APIs, serving containers, feature stores	API gateway, authorized clients	Logging / monitoring zone
Management / Monitoring	SIEM, log aggregators, MLOps dashboards, model registry	Security team workstations, CI/CD pipelines	Training zone (controlled pull), reporting systems

Micro-segmentation takes this further, applying controls at the workload or container level rather than the network subnet level. Cloud-native environments use security groups or network policies that restrict traffic to specific pod labels, reducing the lateral movement an attacker gains if they compromise a single container.

3. Zero Trust Architecture Applied to AI Infrastructure

Zero trust (ZT) is a security paradigm that eliminates implicit trust based on network location. Under classical perimeter security, anything inside the corporate network was presumed safe—a dangerous assumption when AI workloads communicate with cloud services, partner APIs, and remote data pipelines. Zero trust replaces network location with continuous verification: every access request is authenticated, authorized, and encrypted regardless of where it originates.

Applying ZT to AI involves four practical steps. First, enforce identity for every service—not just human users. Each microservice, training job, and inference pod should have a cryptographic identity (SPIFFE/SPIRE or cloud-native workload identity). Second, authorize dynamically: evaluate each request against policy at request time, not at login time. Third, limit blast radius by applying least-privilege at the API scope, dataset, and model level. Fourth, log everything and treat deviations from baseline as signals requiring investigation.

Zero Trust Principle: 'Never trust, always verify' means that even internal services must authenticate and be authorized on every request. Network location—inside the VPC, inside the data center—grants nothing by itself.

4. Firewall Rules: Allow-Listing for AI Endpoints

A default-deny firewall posture means no traffic is permitted unless explicitly allowed. For AI systems, this translates to an allow-list of source IPs or CIDR ranges, protocols, and destination ports. The operational discipline is to start with the minimum working set and open rules only when a business requirement is documented and approved. Undocumented open ports are one of the most common findings in AI infrastructure audits.

- Allow inbound HTTPS (TCP 443) from the API gateway's egress IPs only—no direct client access to inference pods.
- Allow outbound from training jobs to approved data sources (object storage endpoints, licensed data feeds) and nowhere else.
- Block inbound SSH/RDP to AI compute nodes from the public internet; require a bastion or session manager.
- Log all firewall deny events and alert on high-frequency denies from a single source—a sign of reconnaissance.
- Review and audit rules quarterly; remove expired or orphaned rules promptly. Rule bloat accumulates quickly in AI environments that iterate fast.

5. API Gateway: The Hardened Front Door for AI Services

An API gateway consolidates authentication enforcement, rate limiting, request logging, input validation, and routing behind a single entry point. AI services behind the gateway are never exposed directly; all traffic traverses the gateway, which applies policy uniformly. This is operationally advantageous because a security policy change (e.g., blocking a compromised API key) can be applied once at the gateway

rather than patching every service. The gateway also provides a single telemetry source—every request, its latency, its client identity, and its response code, is logged centrally.

Real-World Incident — Optus (2022): Australia's second-largest telecom suffered a breach when a legacy API endpoint was inadvertently exposed to the public internet without authentication. An attacker queried the endpoint iteratively, extracting records for approximately 9.8 million customers including passport and licence numbers. Post-incident analysis found the endpoint had been accessible for an extended period and was not surfaced in monitoring because it was not routed through the corporate API gateway. This incident demonstrates that every AI or data service endpoint must be inventoried and placed behind a controlled gateway—shadow endpoints are discovered by attackers, not defenders.

Selecting an API gateway for AI workloads should consider payload size limits (model inputs can be large), streaming support (for token-by-token LLM responses), and the ability to inspect or log request bodies for abuse detection. Cloud-native options (AWS API Gateway, Azure API Management, Google Cloud Apigee) integrate natively with IAM and provide built-in DDoS mitigation.

6. Encrypting Communications: TLS and Mutual TLS

Transport Layer Security (TLS) encrypts data in transit, preventing eavesdropping and tampering. For AI systems handling sensitive queries or personal data, TLS is mandatory on all channels—client to gateway, gateway to inference service, service to database, and service to logging infrastructure. The exam expects candidates to distinguish standard TLS from mutual TLS (mTLS).

Attribute	TLS (One-Way)	mTLS (Mutual)
Authentication	Server proves identity to client (certificate)	Both server AND client present certificates
Typical Use Case	Public-facing APIs, browser traffic	Service-to-service internal communication
Trust Anchor	Public CA or internal CA for server cert	Internal PKI; each service issued a certificate
Phishing Resistance	No—client not authenticated by cert	Yes—invalid client cert = connection refused
Operational Overhead	Low—standard web server config	Higher—requires PKI, cert lifecycle management

For AI microservices communicating internally, mTLS is the recommended standard. Service mesh solutions (Istio, Linkerd) can automate mTLS between pods without requiring application code changes, significantly reducing the operational burden.

7. Authentication Methods: Matching Strength to Risk

AI APIs serve multiple client types: automated pipelines, human analysts, partner systems, and public users. No single authentication mechanism is appropriate for all. The principle is to match authentication strength to the sensitivity of the resource being accessed.

Method	Best For	Strengths	Weaknesses
API Keys	Simple automated pipelines, public read-only access	Easy to issue and revoke; low client complexity	No built-in expiry; if leaked, grants full key scope; not phishing-resistant
OAuth 2.0 / OIDC	User-facing AI tools, delegated access	Short-lived tokens; scopes limit access; supports refresh	More complex to implement; token leakage still possible
Client Certificates (mTLS)	High-security service-to-service (inference to DB)	Strong mutual identity; phishing-resistant; cert revocation available	PKI infrastructure required; cert rotation overhead
JWT (Bearer Tokens)	Stateless APIs, microservices	Self-contained claims; no session state at server	Cannot revoke until expiry; signing key compromise is catastrophic

8. Rate Limiting and DDoS Protection for AI Endpoints

AI inference endpoints are expensive to serve—a single complex query may consume significant GPU time. Without rate limiting, a single misbehaving client can degrade service for all users, and a targeted DDoS attack can push infrastructure costs to catastrophic levels. Rate limits should be applied at multiple granularities: per API key, per IP address, per user account, and globally per endpoint.

- Per-key limits prevent a single compromised credential from consuming all capacity (e.g., 100 requests/min per key).
- Per-IP limits catch credential-sharing or API key stuffing attacks where one IP cycles through many keys.
- Burst vs. sustained limits: allow brief spikes (burst) but enforce sustained throughput limits over longer windows.
- Exponential backoff responses (HTTP 429 with Retry-After header) help legitimate clients back off gracefully without hammering the API.
- DDoS protection services (AWS Shield, Azure DDoS Protection, Cloudflare) sit in front of the API gateway and absorb volumetric attacks before they reach your infrastructure.
- Inform rate limit decisions with baseline traffic analysis—set limits at 2-3x normal peak, not at an arbitrary number. Too low and you block legitimate users; too high and limits provide no protection.

Real-World Context — AI API Abuse (2023): Multiple AI provider APIs were targeted by credential-stuffing and scraping campaigns in 2023, where attackers obtained leaked API keys from public code repositories (GitHub, Hugging Face) and used them to run large inference workloads, generating thousands of dollars in charges for victims. The attack vector was public exposure of API keys combined with absent or overly generous rate limits. This demonstrates that rate limiting and API key hygiene—scanning repos for exposed secrets—are complementary controls, not alternatives.

9. Authorization: Scopes, RBAC, and Fine-Grained Access

Authentication answers 'who are you?' Authorization answers 'what are you allowed to do?' For AI systems, a robust authorization model is critical because the same API may serve users with very different permission levels: read-only analysts, model fine-tuners, administrators, and automated pipelines.

Scope-based authorization assigns permissions as named capabilities attached to a token. An analyst token might carry 'inference:read' but not 'model:write'. Role-Based Access Control (RBAC) groups permissions into roles aligned to job functions. Attribute-Based Access Control (ABAC) further refines this by evaluating contextual attributes—time of day, data classification, user department—at request time. For AI systems handling personal data under GDPR or HIPAA, ABAC provides the granularity necessary to enforce that only authorized roles can query data about specific individuals.

Model	Authorization Logic	AI Use Case Example
RBAC	Role assigned at provisioning; checked at access	Analyst role = inference only; Admin role = model management + inference
Scope-Based (OAuth)	Token carries named permissions granted at auth time	'inference:read' for querying; 'training:write' for submitting jobs
ABAC	Policy evaluates subject + resource + environment attributes	Sensitive PII dataset accessible only to EU-region users during business hours

10. Input Validation: Stopping Malformed and Malicious Requests

Input validation at the API layer rejects malformed requests before they reach AI model code. For AI APIs, validation is especially important because model inputs are often large, complex, and varied. A validation failure can reveal implementation details to an attacker (verbose errors), trigger unexpected model behavior, or—in the case of prompt injection—manipulate the model's output in unintended ways.

- Schema validation: enforce that request bodies conform to a documented schema (JSON Schema, OpenAPI spec). Reject unexpected fields.
- Type and range checking: ensure numeric fields are within expected bounds; reject strings that are anomalously long (possible prompt injection or buffer manipulation).
- Content filtering at the API gateway: block requests containing known injection patterns or prohibited content categories before they reach the model.
- Return generic error messages to clients on validation failure—never expose schema details, stack traces, or field names in error responses.
- Log all rejected requests with the reason; patterns of validation failures reveal active probing and reconnaissance activity.
- For multimodal AI (vision, audio): validate file types, sizes, and MIME types before passing to the model. Polyglot files can exploit parsers.

11. Network Monitoring and Anomaly Detection for AI Environments

Monitoring transforms reactive incident response into proactive threat detection. AI environments have predictable traffic patterns—training runs occur on a schedule, inference traffic follows usage patterns, administrative access happens during business hours. Deviations from these patterns are strong signals worth investigating. Effective monitoring covers three layers: network flow analysis, API access logs, and host-level telemetry.

- Network flow baselines: record normal traffic volumes, protocols, and source/destination pairs. Alert on new external connections from training nodes, which should never initiate outbound connections.
- API access log analysis: identify unusual query frequencies, rare source IPs, access to endpoints not used in production, and patterns consistent with model extraction (many diverse queries to map decision boundaries).
- Exfiltration detection: large outbound data transfers from inference or data zones should trigger alerts. A legitimate inference response is bounded in size; a bulk data export is not.
- Service account behavior: monitor service accounts for access outside their expected scope. A compromised service account often exhibits scope expansion before a full breach.
- Correlate network events with identity events: a password reset followed immediately by a high-volume API query burst is a red flag worth automated alerting.

12. Defense-in-Depth: Layered Network and API Security

No single control is sufficient. Defense-in-depth stacks multiple overlapping controls so that a failure or bypass at one layer is caught by another. For AI network security, the canonical stack is: DDoS protection → WAF/firewall → API gateway (auth + rate limits + input validation) → mTLS service mesh → fine-grained authorization → anomaly detection and SIEM. An attacker who bypasses rate limiting still faces authentication. An attacker who forges an authentication token still faces RBAC authorization checks. An attacker who obtains valid credentials still triggers anomaly detection if their behavior deviates from the baseline.

Defense-in-Depth: Layer controls so that each independent layer must be bypassed separately. The goal is not to make any single layer impenetrable but to ensure that the combination of layers raises the cost and complexity of an attack beyond what most adversaries are willing to invest.

On the exam, defense-in-depth questions test whether candidates can identify which layer a given control belongs to and what residual risk remains if that layer is absent. Practice mapping each control (e.g., mTLS, rate limiting, input validation) to the layer it occupies and the attack it mitigates.

Key Terms Glossary

Term	Definition
Network Segmentation	Dividing a network into isolated zones with controlled inter-zone traffic, limiting lateral movement.
Zero Trust Architecture (ZTA)	Security model requiring continuous verification of every access request regardless of network location.

API Gateway	A single, hardened entry point for API traffic that enforces authentication, rate limiting, and logging.
mTLS (Mutual TLS)	TLS variant where both client and server authenticate using certificates, enabling phishing-resistant service-to-service identity.
Rate Limiting	Restricting the number of API requests a client can make within a time window to prevent abuse and resource exhaustion.
DDoS (Distributed Denial of Service)	Overwhelming a target with traffic from many sources to render it unavailable; mitigated by upstream scrubbing services.
RBAC (Role-Based Access Control)	Authorization model assigning permissions to roles; users or services inherit permissions from their assigned role.
ABAC (Attribute-Based Access Control)	Authorization model evaluating subject, resource, and environmental attributes against policy at request time.
OAuth 2.0 / OIDC	Delegation framework (OAuth 2.0) and identity layer (OIDC) enabling scoped, token-based access without sharing credentials.
Input Validation	Verifying that API request contents conform to expected schema, type, and value constraints before processing.
Defense-in-Depth	Security strategy applying multiple overlapping controls so that bypassing one layer does not yield full access.
East-West Traffic	Traffic flowing between services within a data center or cloud environment, as opposed to north-south (external) traffic.
SPIFFE / SPIRE	Open standards for issuing cryptographic workload identities in cloud-native environments, enabling zero-trust service authentication.
Model Extraction Attack	Adversarial technique of querying an inference API repeatedly to reconstruct the underlying model's behavior or parameters.

Quick Revision Checklist

- AI network security requires segmentation, zero trust, API gateway controls, and monitoring — not just a perimeter firewall.
- Segment into training, inference, and management zones; restrict east-west traffic between zones.
- Zero trust: authenticate and authorize every request, even from inside the network; location grants nothing.
- Use an API gateway as the single entry point; never expose inference services directly to clients.
- Encrypt all traffic with TLS; use mTLS for service-to-service communication to provide mutual authentication.
- Match authentication strength to risk: API keys for simple pipelines, OAuth for user-facing tools, certificates for high-security service links.
- Apply rate limits at key, IP, and global granularities; use DDoS protection services in front of the gateway.
- Authorization (RBAC/ABAC/scopes) is distinct from authentication: always enforce what a verified caller is permitted to do.
- Validate all inputs at the API layer; reject malformed requests before they reach the model.

- Monitor network flows and API logs for baseline deviations; anomalies are indicators of reconnaissance, exfiltration, or compromise.
- Defense-in-depth stacks DDoS protection, firewall, API gateway, mTLS, authorization, and monitoring — failure of one layer is caught by another.
- Exam tip: map each control (segmentation, rate limiting, mTLS, input validation) to the attack it mitigates and the layer it occupies.

10 Exam-Based MCQs — Network and API Access Controls

Test yourself after reading. These questions are written in real exam style — scenario-heavy, with plausible distractors. Read every explanation, including for questions you answered correctly.

Q1. Scenario: A post-incident review at a mid-size AI company reveals that a legacy model inference endpoint was accessible from the public internet without authentication. The endpoint was not logged anywhere, not routed through the API gateway, and had been exposed for eight months. Customer data was exfiltrated before detection. The CISO asks the security team to identify the highest-priority remediation. What should the security team recommend FIRST?

- A) Deploy a WAF in front of all inference endpoints immediately
- B) Inventory all AI API endpoints and ensure each is routed through the corporate API gateway
- C) Enable TLS on the training database
- D) Implement ABAC policies for all existing access tokens

Answer: B **Explanation:** B is correct because the Optus-style incident demonstrates that undiscovered endpoints accessible outside the gateway are the highest-risk gap. Inventorying and gating all endpoints through the gateway addresses the root cause. A is wrong because a WAF helps but only works if traffic reaches it — ungated shadow endpoints bypass it. C is wrong because encrypting the training database, while valuable, does not address the immediate exposure of public-facing API endpoints. D is wrong because ABAC is a fine-grained authorization improvement; it cannot protect endpoints that are entirely unauthenticated and publicly accessible.

Q2. Scenario: Developers at a cybersecurity firm accidentally commit their AI inference API keys to a public GitHub repository. Within hours, automated bots discover the keys and begin sending thousands of inference requests per minute, generating thousands of dollars in cloud costs. The firm's incident response team needs to identify which controls would have prevented or minimized the financial impact. Which combination of controls BEST prevents the attack described?

- A) Rate limiting per API key and DDoS protection at the network edge
- B) Input validation and WAF rules blocking known injection patterns
- C) mTLS between inference pods and the model registry
- D) ABAC policies restricting dataset access by user department

Answer: A **Explanation:** A is correct because the attack described is credential-stuffing combined with high-volume inference abuse — exactly the scenario that rate limiting per key and DDoS protection are designed to mitigate. Rate limits cap the damage from any single leaked key; DDoS protection absorbs volumetric floods. B is wrong because input validation addresses malformed requests and injection, not

high-volume legitimate-looking requests using valid credentials. C is wrong because mTLS secures service-to-service identity; it does not address client-facing API abuse via exposed keys. D is wrong because ABAC controls authorization scope but does not limit request volume or prevent use of a legitimately scoped but leaked credential.

Q3. Scenario: A threat model review at an AI security company identifies that a compromised inference pod could potentially make direct network connections to the training data lake and model registry in the same cloud VPC. The security architect needs to recommend an architectural control that prevents this lateral movement scenario. Which architecture decision BEST addresses the security concern identified?

- A) Deploy a service mesh with mTLS between all training and inference pods
- B) Apply network segmentation to isolate the training zone and restrict east-west traffic
- C) Implement RBAC so only model administrators can push to the model registry
- D) Enable audit logging on the model registry API

Answer: B Explanation: B is correct because the threat is lateral movement from a compromised inference pod to the training environment. Network segmentation with restricted east-west traffic between the inference zone and the training zone is the primary control that prevents this. A is wrong because mTLS provides authenticated communication but does not prevent an authorized pod (or one that has compromised a valid identity) from reaching the training zone if routing permits it; segmentation denies the routing. C is wrong because RBAC on the registry addresses who can push models but not whether an attacker who compromised an inference pod can reach the training zone at the network level. D is wrong because logging detects and records the lateral movement after the fact but does not prevent the attacker from reaching the training data.

Q4. Which statement BEST distinguishes authentication from authorization in the context of AI API security?

- A) Authentication encrypts API traffic; authorization decrypts it
- B) Authentication verifies the caller's identity; authorization determines what the verified caller is permitted to do
- C) Authentication is enforced at the API gateway; authorization is enforced at the firewall
- D) Authentication applies to human users only; authorization applies to service accounts

Answer: B Explanation: B is correct and is the canonical definition. Authentication answers 'who are you?' via credentials or certificates; authorization answers 'what are you allowed to do?' via policy evaluation. A is wrong because encryption is a separate control (TLS) unrelated to authentication or authorization. C is wrong because both authentication and authorization can be enforced at the API gateway; the distinction is functional, not architectural. D is wrong because both authentication and authorization apply to human users AND service accounts — the distinction is not based on principal type.

Q5. What is the PRIMARY security advantage of mTLS over standard TLS for service-to-service AI communication?

- A) mTLS uses stronger encryption algorithms than standard TLS

- B) mTLS eliminates the need for an API gateway by securing all communication channels
- C) mTLS requires both the client and server to authenticate using certificates, preventing unauthorized services from connecting
- D) mTLS automatically rotates encryption keys, reducing the window of exposure if a key is compromised

Answer: C **Explanation:** C is correct. The defining feature of mTLS is mutual authentication — both parties present certificates. This means a service without a valid certificate cannot establish a connection, preventing unauthorized services (including attacker-controlled ones) from impersonating legitimate callers. A is wrong because mTLS and TLS use the same cipher suites; the difference is in authentication directionality, not encryption strength. B is wrong because mTLS secures the transport channel; an API gateway serves a different function (routing, rate limiting, logging) and is complementary, not redundant. D is wrong because key rotation is a PKI management practice separate from the mTLS handshake mechanism.

Q6. Which of the following BEST describes the role of an API gateway in an AI security architecture?

- A) It replaces firewalls by performing deep packet inspection at layer 3
- B) It provides a single hardened entry point that enforces authentication, rate limiting, and logging for all AI API traffic
- C) It encrypts AI model weights during transit to prevent intellectual property theft
- D) It acts as a backup inference node if the primary model server fails

Answer: B **Explanation:** B is correct. An API gateway's primary security functions are centralizing policy enforcement (authentication, authorization, rate limiting) and providing a single telemetry point for all API traffic, while keeping backend services unexposed. A is wrong because API gateways operate at application layer (layer 7), not network layer 3; they complement rather than replace firewalls. C is wrong because model weight encryption in transit is handled by TLS on the transport layer, not by the API gateway specifically. D is wrong because API gateways are security and routing infrastructure; high-availability failover for inference is a separate availability concern.

Q7. Scenario: A security audit of an AI company's infrastructure finds that all traffic originating from the internal corporate subnet is automatically trusted by the AI inference service, with no authentication required. The justification given is that the subnet is 'inside the firewall, so it's safe.' Which zero trust principle is violated by the configuration described?

- A) Least privilege
- B) Assume breach
- C) Verify explicitly
- D) Use of encryption

Answer: C **Explanation:** C is correct. 'Verify explicitly' means every request must be authenticated and authorized regardless of source. Trusting all traffic from an internal subnet without verification violates this core zero trust principle — network location is not a substitute for identity verification. A is wrong because least privilege concerns the scope of permissions granted, not whether authentication is performed. B is

wrong because 'assume breach' means designing for containment after a compromise; the described configuration relates to verification before access is granted. D is wrong because encryption addresses confidentiality and integrity of data in transit; the described issue is about authentication, not encryption.

Q8. An AI company wants to enforce that a specific service account can query the fraud detection model but cannot modify model weights or access training data. Which access control model is MOST appropriate?

- A) Discretionary Access Control (DAC) — the service account owner sets its own permissions
- B) Role-Based Access Control (RBAC) with a 'fraud-analyst-reader' role scoped to inference only
- C) Mandatory Access Control (MAC) based on data classification labels
- D) Single sign-on (SSO) with multi-factor authentication

Answer: B **Explanation:** B is correct. RBAC allows the creation of a role with precisely the permissions needed — inference read access to the fraud model — and denies everything else. The service account is assigned that role and inherits only those permissions. A is wrong because DAC allows resource owners to set permissions, which introduces inconsistency and relies on individual owners applying least privilege correctly — not a reliable enterprise model. C is wrong because MAC is classification-driven and appropriate for data sensitivity tiering, but it does not by itself define the specific action-level permissions (query vs. modify) needed here. D is wrong because SSO and MFA address authentication strength, not authorization scope — they verify identity, not what the verified identity is allowed to do.

Q9. Which of the following network monitoring observations is MOST likely to indicate a model extraction attack in progress?

- A) A single source IP sending 50 requests per minute consistently over several hours
- B) High-volume inference traffic from a single API key covering an unusually diverse range of input types, with queries designed to systematically probe decision boundaries
- C) Periodic spikes in inference traffic aligned with business hours in a known customer time zone
- D) A service account making mTLS connections to the inference API from within the management zone

Answer: B **Explanation:** B is correct. Model extraction involves systematically querying an inference API to reconstruct the model's decision logic. The signature is diverse, structured inputs designed to map decision boundaries — not random traffic but methodical probing that covers the input space. A is wrong because 50 requests per minute is moderate traffic that could indicate a legitimate automated pipeline; without evidence of systematic input diversity, it is not specific to model extraction. C is wrong because traffic spikes aligned with business hours suggest normal usage patterns, not adversarial activity. D is wrong because a service account connecting from within the management zone via mTLS is expected and appropriate operational behavior for authorized services.

Q10. A security engineer is designing the network architecture for a new AI training platform. Which sequence BEST represents a defense-in-depth approach for inbound API traffic?

- A) Client → TLS encryption → model inference → API gateway → SIEM

- B) Client → DDoS protection → API gateway (auth + rate limit + validation) → mTLS → inference service → authorization check
- C) Client → firewall → model inference → rate limiting → logging
- D) Client → API gateway → authentication → public inference endpoint → network segmentation

Answer: B **Explanation:** B is correct. This sequence places controls in logical order: DDoS scrubbing handles volumetric attacks before traffic reaches application infrastructure; the API gateway applies authentication, rate limiting, and input validation; mTLS secures the internal hop; authorization is checked at the service level for fine-grained control. Each layer addresses a different attack class. A is wrong because TLS is a transport control applied throughout, not a discrete gateway layer, and the model inference step appears before the API gateway — bypassing all gateway controls. C is wrong because rate limiting after the firewall but before authentication allows unauthenticated traffic to reach the inference service, and the ordering omits authentication entirely. D is wrong because placing a 'public inference endpoint' after the API gateway defeats the purpose of the gateway — inference services should never be directly publicly accessible.